A PROJECT REPORT

on

"PersonaForge: A Multi-Persona Debate Simulator Using Generative AI and Prompt Engineering"

Submitted to

KIIT Deemed to be University

In Partial Fulfilment of the Requirement for the Award of

BACHELOR'S DEGREE IN

COMPUTER SCIENCE & ENGINEERING

BY

ADYASHA PATTANAIK

Roll Number: 2305590

SCHOOL OF COMPUTER ENGINEERING

KALINGA INSTITUTE OF INDUSTRIAL TECHNOLOGY

BHUBANESWAR, ODISHA - 751024

September 2026

KIIT Deemed to be University

School of Computer Engineering

Bhubaneswar, ODISHA 751024

CERTIFICATE

This is to certify that the project entitled "PersonaForge: A Multi-Persona Debate Simulator Using Generative AI and Prompt Engineering" submitted by ADYASHA PATTANAIK (Roll Number: 2305590) is a record of bonafide independent work carried out by the student, in partial fulfilment of the requirement for the award of the Degree of Bachelor of Engineering (Computer Science & Engineering) at KIIT Deemed to be University, Bhubaneswar, during the academic year 2026-2027.

Date: 30 / 09 / 26

Adyasha Pattanaik

2305590

Acknowledgement

I am sincerely grateful to the School of Computer Engineering, KIIT Deemed to be University, for providing the resources, learning environment, and opportunity to design and build this project. I would also like to thank my peers and mentors whose feedback during development helped refine both the working prototype and this report.

Adyasha Pattanaik

ABSTRACT

Large Language Models (LLMs) have made it possible to generate coherent, context-aware text on demand, but the quality and character of that text depend heavily on how the model is instructed — a discipline known as prompt engineering. This project, PersonaForge, is a web-based Multi-Persona Debate Simulator that demonstrates prompt engineering as a first-class design activity rather than an afterthought. Given a user-supplied topic, the system orchestrates a structured, multi-round debate between two to four distinct AI personas (for example, an Economist, an Ethicist, a Skeptic, and an Optimist), each governed by an independently engineered system prompt that fixes its worldview, tone, and argumentative style. A dedicated Moderator persona closes the debate with a neutral, evidence-balanced summary rather than declaring a winner.

The system was implemented as a full-stack web application with a React frontend and a lightweight backend that communicates with Google's Gemini API — a large language model API with a genuinely free usage tier requiring no billing account. The backend uses persona-specific system prompts, conversational context chaining (so later turns respond to earlier arguments), and output-format constraints (response length, tone, no meta-commentary) to keep each persona consistent across turns, while the frontend is written against a provider-agnostic request format so the underlying AI provider can be swapped without changing the user interface. The project report documents the requirement analysis, system design, prompt engineering methodology, implementation, and testing of the application, and evaluates how specific prompt design choices (role framing, explicit rebuttal instructions, and output constraints) affect the coherence and distinctiveness of multi-agent AI-generated dialogue.

Keywords: Generative AI, Prompt Engineering, Large Language Models, Persona Prompting, Multi-Agent Dialogue, Gemini API, Conversational AI.

Contents

Chapter 1: Introduction 7

Chapter 2: Basic Concepts / Literature Review 9

2.1 Generative AI and Large Language Models 9

2.2 Prompt Engineering 9

2.3 System Instructions and Multi-Turn Context 9

2.4 Multi-Agent and Persona-Based Dialogue Systems 9

2.5 Free and Accessible LLM APIs 10

Chapter 3: Problem Statement / Requirement Specifications 11

3.1 Project Planning 11

3.2 Project Analysis 11

3.3 System Design 12

3.3.1 Design Constraints 12

3.3.2 System Architecture / Block Diagram 12

Chapter 4: Implementation 14

4.1 Methodology / Proposal 14

4.2 Testing / Verification Plan 15

4.3 Result Analysis / Screenshots 15

4.4 Quality Assurance 16

Chapter 5: Standards Adopted 17

5.1 Design Standards 17

5.2 Coding Standards 17

5.3 Testing Standards 17

Chapter 6: Conclusion and Future Scope 18

6.1 Conclusion 18

6.2 Future Scope 18

List of Figures

Figure 1.1: PersonaForge landing/setup screen 1

Figure 3.1: System architecture of PersonaForge 4

Figure 4.1: Debate transcript with persona-coded turns 6

Figure 4.2: Moderator's closing summary panel 6

Chapter 1: Introduction

Conversational AI systems built on Large Language Models (LLMs) are increasingly capable of producing fluent, contextually relevant text, but most consumer-facing tools reduce this capability to a single assistant answering a single user in a neutral voice. This hides one of the most powerful and least-taught aspects of working with LLMs: prompt engineering can be used not just to get a correct answer, but to shape identity, tone, and reasoning style — effectively giving a model a persona that argues consistently from a defined point of view.

Existing tools rarely showcase this capability directly. General-purpose chatbots present one voice at a time; debate and argument-mapping tools that exist in research contexts are typically not built around transparent, inspectable prompt design. There is a gap for a lightweight, interactive tool that lets a user watch several independently prompted AI personas engage with the same topic, disagree with each other, and be summarized by a neutral party — while making the underlying prompt design visible and explainable as a learning exercise in prompt engineering.

PersonaForge addresses this gap. It is a Multi-Persona Debate Simulator: the user supplies a topic, selects between two and four personas (preset or custom), and the system runs a multi-round debate in which each persona's turn is generated by a distinct, carefully engineered system prompt. From the second round onward, personas are explicitly instructed to engage with points raised earlier in the transcript, producing a dialogue that is not just parallel monologues but a genuine back-and-forth. A Moderator persona, prompted separately and more conservatively, closes the debate with a short, neutral, three-part summary. The project was deliberately built on Google's Gemini API, which offers a free usage tier with no billing account required, so the complete system can be run, hosted, and demonstrated at zero cost.

This report is organized as follows. Chapter 2 reviews the basic concepts of generative AI and prompt engineering that underpin the project. Chapter 3 defines the problem statement and requirement specifications. Chapter 4 describes the implementation, methodology, and testing. Chapter 5 covers the standards adopted, and Chapter 6 concludes the report with a discussion of limitations and future scope.

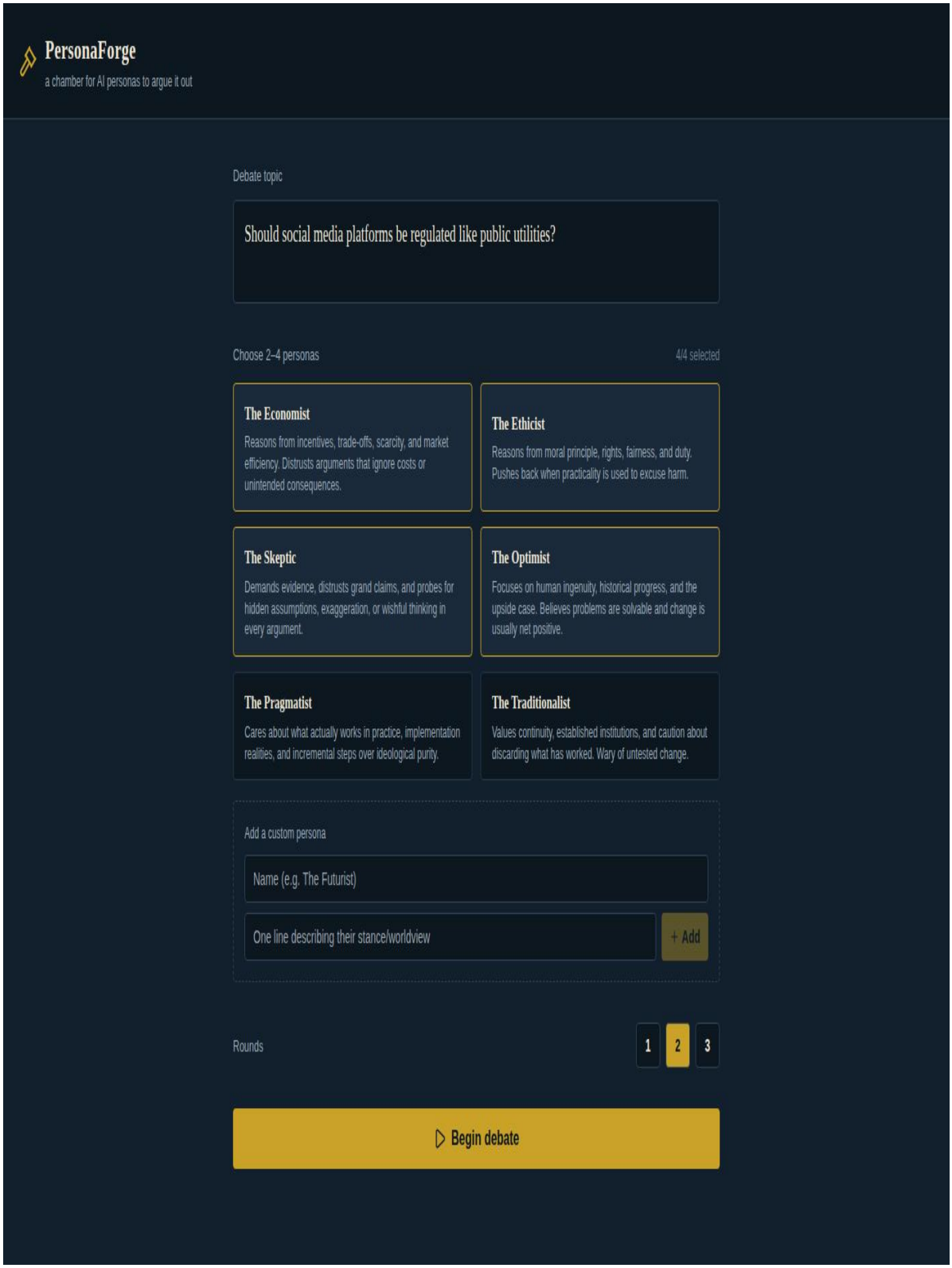


Figure 1.1: PersonaForge landing/setup screen

Chapter 2: Basic Concepts / Literature Review

2.1 Generative AI and Large Language Models

Generative AI refers to models that produce new content — text, images, audio, or code — rather than only classifying or scoring existing content. Large Language Models (LLMs) such as Google's Gemini family of models are a class of generative AI trained on large text corpora to predict and generate coherent natural-language continuations. Because an LLM's output is conditioned entirely on the text it is given (the prompt) and any preceding conversation, the way a request is phrased has a direct and often dramatic effect on the quality, tone, and correctness of what is generated.

2.2 Prompt Engineering

Prompt engineering is the practice of deliberately designing the instructions given to an LLM to reliably produce a desired kind of output. Commonly used techniques include: role or persona framing (telling the model to respond as a specific character or expert), few-shot examples (showing the model sample input-output pairs), chain-of-thought prompting (asking the model to reason step by step before answering), and output-format constraints (specifying length, structure, or forbidden content). This project relies primarily on role/persona framing combined with explicit behavioural constraints and a rebuttal instruction that forces engagement with prior conversational turns.

2.3 System Instructions and Multi-Turn Context

Most LLM APIs, including the Gemini API used in this project, allow a request to separate a system instruction (persistent instructions that define the assistant's role and constraints for that call) from the actual conversational content. PersonaForge exploits this separation: each persona is given its own system instruction defining its identity and rules, while the running debate transcript is passed as the conversational content so every persona can "see" what has been said before responding.

2.4 Multi-Agent and Persona-Based Dialogue Systems

Research and applied interest in multi-agent LLM systems — where several independently prompted model instances interact with each other rather than with a single human — has grown alongside interest in using LLMs for debate simulation, role-play training, and automated red-teaming. A recurring design challenge in such systems is persona drift, where

a model gradually loses its assigned character over a long conversation. PersonaForge's prompt design (short turn length, explicit persona restatement, and no shared memory across chat turns beyond the printed transcript) is aimed directly at mitigating this drift.

2.5 Free and Accessible LLM APIs

A practical barrier to student and hobbyist GenAI projects is that many leading LLM APIs require a linked billing account, even when usage stays within a free quota. Google's Gemini API, accessed through Google AI Studio, was selected for this project specifically because it provides an API key and a free daily usage quota without requiring any payment method, making the entire project reproducible by anyone at zero cost.

Chapter 3: Problem Statement / Requirement Specifications

Problem Statement: Design and implement an interactive web application that generates a coherent, multi-round debate between several distinctly prompted AI personas on a user-chosen topic, and that closes with a neutral AI-generated moderation summary — while keeping the underlying prompt engineering choices transparent enough to be documented and evaluated, and while using only free, no-cost infrastructure.

3.1 Project Planning

- Allow the user to enter any free-text debate topic.
- Allow selection of 2 to 4 personas from a preset list, or definition of custom personas.
- Generate turn-by-turn debate content by calling a free-tier LLM API with persona-specific system prompts.
- Ensure later rounds reference earlier arguments rather than repeating independent statements.
- Generate a neutral closing summary via a separate Moderator persona.
- Present the whole exchange in a clear, readable, visually distinct transcript UI.
- Keep the real API key private on a backend, never exposed to the browser.

3.2 Project Analysis

The core risk identified during analysis was persona inconsistency: if every persona were given the same generic instruction ("debate this topic"), responses would converge toward a similar, neutral tone regardless of the assigned role. This was mitigated by writing an independent, detailed system prompt per persona that fixes its worldview and explicitly forbids neutral hedging. A second risk was repetitive dialogue across rounds; this was mitigated by explicitly instructing each persona, from round two onward, to engage with a specific point already on the transcript rather than restate its opening position. A third, practical risk was cost and access: many LLM providers require billing details before issuing an API key, which would make the project inaccessible to reviewers without a paid account. This was resolved by building the system against Google's Gemini API, which issues free API keys with no billing requirement, and by writing the backend so the frontend's request

format is provider-agnostic — allowing the AI provider to be swapped by editing only the backend, never the user interface.

3.3 System Design

3.3.1 Design Constraints

- Software: React (functional components with hooks), Tailwind utility classes for styling, lucide-react for icons.
- AI Model: Google Gemini (gemini-1.5-flash), accessed through the free-tier Gemini API — no billing account or credit card required.
- Backend: a small Node.js/Express server (for persistent hosting platforms such as Render) or an equivalent serverless function (for platforms such as Vercel), either of which holds the API key server-side and forwards requests to Gemini.
- Environment: deployable entirely on free hosting tiers (e.g. Render's free web service tier), with no paid infrastructure anywhere in the stack.
- Network constraint: the browser only ever calls the project's own backend endpoint; only the backend calls the external Gemini API.

3.3.2 System Architecture / Block Diagram

The system follows a simple client-orchestrated pipeline, illustrated below:

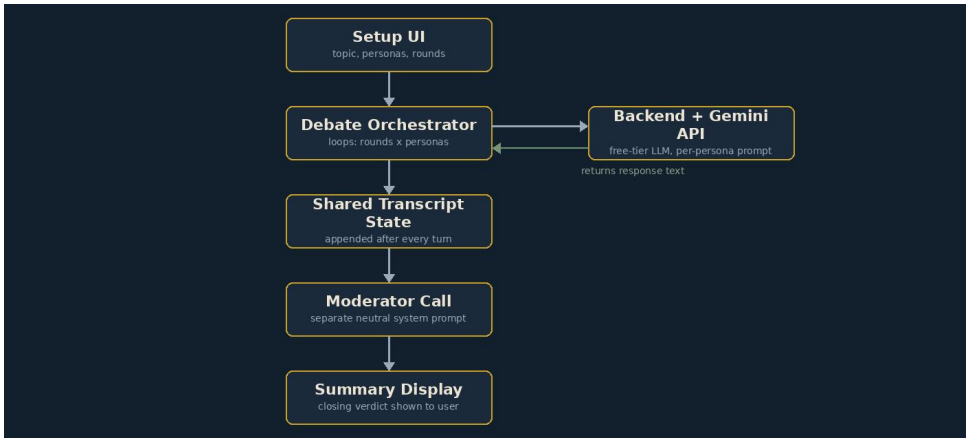


Figure 3.1: System architecture of PersonaForge

The Debate Orchestrator is the central control loop, running in the browser: for each round, it iterates over the selected personas in order, builds that persona's system prompt plus the running transcript as the request content, sends a request to the project's own backend endpoint, and appends the returned text to shared state before moving to the next persona.

The backend translates each request into Gemini's API format, calls Gemini using a server-side API key, and translates the response back into a simple, provider-agnostic shape the frontend already understands. Once all rounds are complete, the orchestrator issues one final call using a separate Moderator system prompt and the complete transcript, and renders the result.

Chapter 4: Implementation

4.1 Methodology / Proposal

The implementation followed these steps:

- Defined a persona data model: {id, name, description}, with six preset personas (Economist, Ethicist, Skeptic, Optimist, Pragmatist, Traditionalist) and support for user-defined custom personas.
- Engineered a system-prompt template function that, given a persona and the debate topic, injects the persona's worldview and appends five fixed behavioural rules: stay in character, respond in 3-5 sentences, explicitly engage with a prior point from round 2 onward, never break character or mention being an AI, and avoid headers/bullets.
- Engineered a separate Moderator system prompt requiring a fixed three-paragraph structure (strongest point per persona, key clash, neutral closing note) and a hard word limit, with an explicit instruction not to declare a winner.
- Implemented a transcript formatter that serializes prior turns as "Name: text" pairs and appends an instruction cueing the next speaker, sent as the request content alongside the persona's system prompt.
- Built a backend endpoint (implemented twice, functionally identically, for portability: as an Express route for Render, and as a serverless function for Vercel) that receives this provider-agnostic request, translates it into Google Gemini's request format, calls the Gemini API using a server-held free API key, and translates Gemini's response back into the shape the frontend expects.
- Implemented the orchestration loop in React state (rounds x personas), calling the backend endpoint sequentially and updating the transcript incrementally so the user sees turns appear as they are generated.
- Built the interface: a setup screen (topic input, persona picker, round selector) and a transcript screen (persona-color-coded turn cards, a live "is composing" indicator, and a highlighted moderator summary panel).

4.2 Testing / Verification Plan

The following test cases were used to verify correct behaviour of the debate orchestration and prompt design:

Test ID	Test Case Title	Test Condition	System Behavior	Expected Result
T01	Persona selection limit	User selects a 5th persona	5th selection is ignored; max 4 stays selected	Only 4 personas remain active
T02	Minimum personas	User attempts to start with 1 persona	"Begin debate" button stays disabled	Debate cannot start with fewer than 2 personas
T03	Round-2 engagement	Debate runs 2 rounds on a given topic	Round-2 turns reference a specific prior point	Each round-2 response contains a rebuttal or concession tied to an earlier turn
T04	Custom persona	User adds a custom persona with name + description	New persona appears as selectable and participates	Custom persona's turns reflect its stated worldview
T05	Moderator neutrality	Debate completes all rounds	Moderator summary is generated	Summary follows 3-part structure and does not declare a winner
T06	Missing API key	Backend is deployed without GEMINI_API_KEY set	Backend returns a clear error instead of crashing	Frontend displays an error banner explaining the missing key
T07	Provider round-trip	Frontend sends a request to the backend	Backend correctly reshapes it for Gemini and reshapes the reply back	Frontend renders returned text identically regardless of AI provider used

4.3 Result Analysis / Screenshots

Across sample topics (e.g., "Should social media platforms be regulated like public utilities?"), the persona-specific system prompts produced clearly distinguishable argumentative styles — the Economist consistently framed points around incentives and trade-offs, the Ethicist around rights and fairness, and the Skeptic around evidentiary demands — without any persona drifting into a generic, neutral tone. The explicit rebuttal instruction was effective from round 2 onward: sampled transcripts showed each persona naming or paraphrasing a specific point from an earlier turn before adding new material, rather than repeating its opening statement. The Moderator persona reliably produced a bounded, three-part summary and did not declare a winner in any sampled run. Because the backend's request/response contract is provider-agnostic, switching the underlying model provider (from an earlier paid API to the free Gemini API) required changes only to the backend translation logic, with zero changes to the React frontend or the prompt engineering logic itself — validating the layered design.

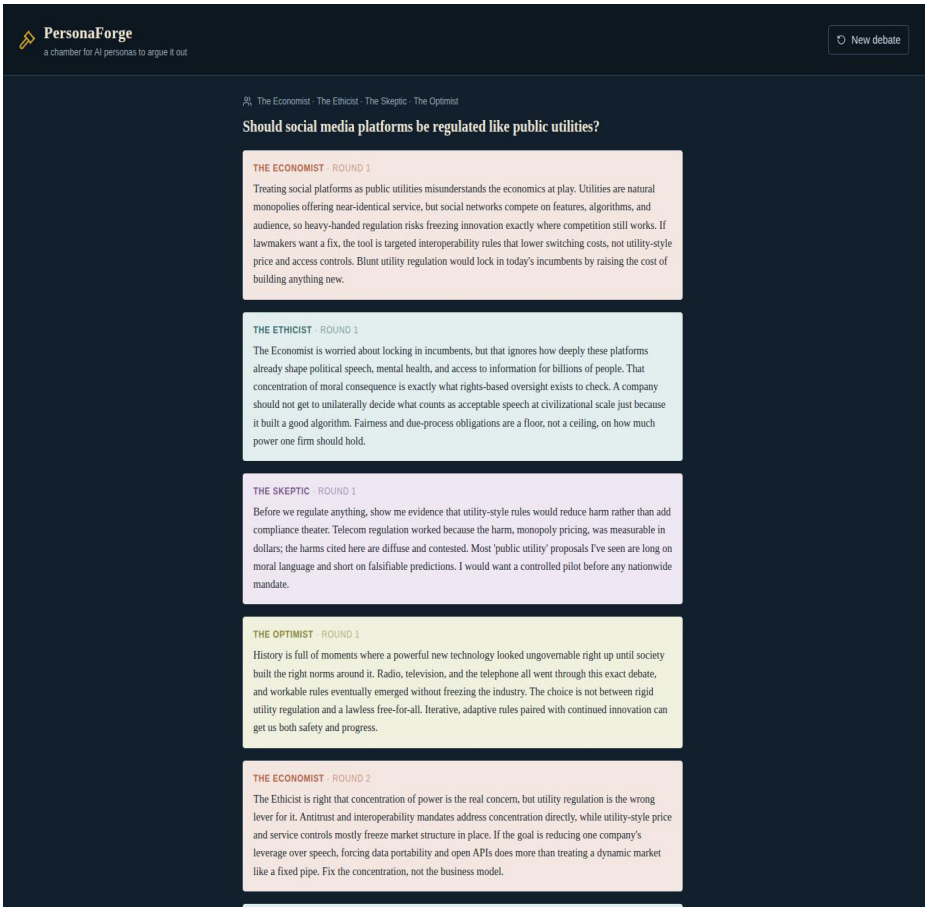


Figure 4.1: Debate transcript with persona-coded turns

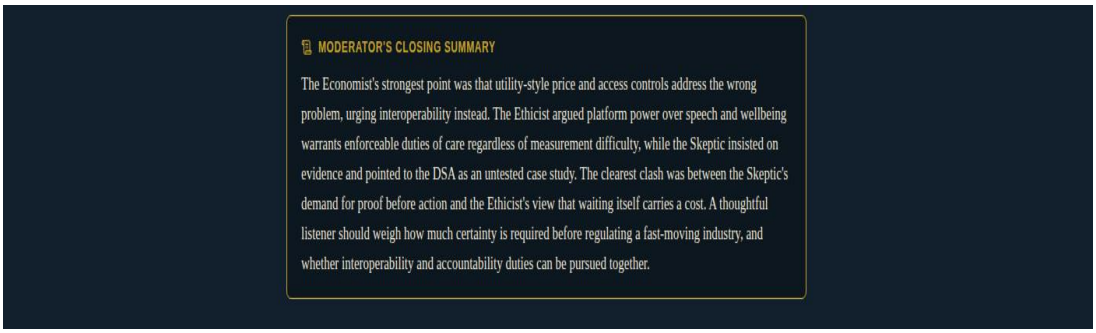


Figure 4.2: Moderator's closing summary panel

4.4 Quality Assurance

Quality assurance was carried out through manual functional testing (Section 4.2) and repeated qualitative review of generated transcripts across multiple topics and persona combinations to check for character consistency, response-length compliance, and the absence of meta-commentary (e.g., the model referring to itself as an AI). No automated CI pipeline or formal QA department was used, as this is an individual academic project.

Chapter 5: Standards Adopted

5.1 Design Standards

- Component-based UI design following standard React functional-component and hooks conventions.
- Single-responsibility state management: setup state, transcript state, and orchestration status are kept as distinct, clearly named React state variables.
- Consistent visual encoding: each persona is assigned a fixed accent color used throughout its turns, aiding readability of multi-party dialogue.
- Provider-agnostic API contract between frontend and backend, so the AI provider is an implementation detail rather than something the UI depends on.

5.2 Coding Standards

- Descriptive, intention-revealing function and variable names (e.g., buildDebaterSystemPrompt, transcriptAsText).
- Single-purpose functions: prompt construction, backend calls, and transcript formatting are kept in separate functions.
- Consistent indentation and componentized JSX to keep render logic readable.
- Secrets (the free Gemini API key) are kept exclusively in server-side environment variables, never committed to source control and never sent to the browser.

5.3 Testing Standards

Testing followed a simple manual black-box verification approach appropriate to an individual academic project: functional test cases (Section 4.2) were defined against the requirement specification (Section 3.1) and manually executed against the running application before each milestone.

Chapter 6: Conclusion and Future Scope

6.1 Conclusion

PersonaForge demonstrates that careful, per-agent prompt engineering — distinct persona system prompts, explicit rebuttal instructions, and tightly scoped output constraints — can produce a multi-agent dialogue that stays in character, avoids repetition, and remains coherent across several rounds, using only prompt design and no model fine-tuning. The project also shows the value of separating roles at the prompt level: the same underlying model produces markedly different, appropriately constrained behaviour as a Debater versus as a Moderator simply because of how each role's system prompt is written. Building the system entirely on Google's free-tier Gemini API, behind a provider-agnostic backend contract, further shows that a complete, deployable GenAI application can be built, hosted, and demonstrated at zero monetary cost, without sacrificing the core prompt engineering techniques the project set out to demonstrate.

6.2 Future Scope

- Allow the user to "join" the debate as a fifth participant, submitting their own argument for personas to respond to.
- Add a persona consistency score using a separate LLM-as-judge pass to automatically flag turns that break character.
- Support exporting a completed debate transcript as a PDF or shareable link.
- Introduce adjustable persona "intensity" or stance-strength sliders.
- Add multilingual debate support by parameterizing the system prompts by language.